

Restro Order System

Node.js restaurant ordering platform (mobile order · table order · kitchen display · POS · analytics) — progress notes and detailed remaining-work plan.

1. Where the project stands right now

The backend and the customer-facing ordering experience are built and working end-to-end. The kitchen display, the admin/POS dashboard, and the supporting docs are the remaining pieces.

Area	Status	Notes
Project scaffold (package.json, folders)	Done	Express + Socket.IO + JSON data store, no native DB build step
Data layer (db.js) + seed data	Done	Categories, items, options/toppings, tables, orders, staff calls
REST API (server.js)	Done	Menu, items, categories, tables, orders, staff calls, analytics, admin login
Realtime events (Socket.IO)	Done on backend	new_order, order_updated, staff_call, staff_call_updated, menu_updated
Design system (style.css)	Done	Izakaya-inspired palette, type, buttons, cards, sheets, toasts
Translations (i18n.js)	Done	EN / NE / JA strings for customer-facing UI; kitchen/admin strings not started
Landing page (index.html)	Done	Table picker + links to kitchen/admin
Customer order page + logic	Done	Menu browse, options/toppings, cart, place order, staff call, live order tracking
Kitchen display page	Not started	Ticket board, status transitions, timers
Admin / POS dashboard	Not started	Orders, menu mgmt, tables, staff calls, analytics charts, billing
README / setup instructions	Not started	How to run, folder guide, feature-to-file map
Cross-feature polish + testing	Not started	Mobile QA, empty/error states, accessibility pass

2. Remaining work, in detail

2.1 Kitchen display (public/kitchen.html + public/js/kitchen.js)

Covers the "Kitchen / Order Transmission" feature group — the board kitchen staff watch to see incoming orders and move them through prep.

- Dark, high-contrast ticket board (readable from a distance) rendered as a responsive grid of order cards.
- Each ticket shows: table number, order id, line items with quantity/options/notes, and an elapsed-time badge.
- Colour-coded urgency: green under ~5 min, amber 5–12 min, red past 12 min, recalculated on a timer.
- Tap-through status flow per ticket: Pending → Cooking → Ready → Served, using PATCH /api/orders/:id.
- Live updates via Socket.IO (new_order, order_updated) — no polling required, board updates instantly when a customer or waiter submits an order.
- Sold-out shortcut: mark an item sold out directly from a ticket if the kitchen runs out mid-service (PATCH /api/items/:id).
- Optional: simple audio/visual pulse on new-order arrival, and a filter to show only a given station/category.

2.2 Admin / POS dashboard (public/admin.html + public/js/admin.js)

This single dashboard covers several USEN feature groups at once: POS/Accounting, Payment, Sales Analytics, Restaurant-specific Automation, and menu/table administration.

a) PIN login

- Simple PIN gate using POST /api/admin/login (seed PIN: 1234) — stores a demo session token in localStorage.
- Not production-grade auth; documented as a placeholder for real staff-account login + roles (owner / cashier / kitchen).

b) Orders & billing (POS)

- Live order queue with status filters (pending / cooking / ready / served / paid) backed by GET /api/orders.
- Table view: tap a table to see its open orders combined into one bill, matching "table-based billing" and "combined bill".
- Mark an order Paid (PATCH status → paid) to simulate POS/payment capture; total, item breakdown, and timestamp are already stored per order.
- Split-bill and discount are noted as a follow-up: the data model supports adding a discount field and a "split into N shares" helper on the client without backend changes.
- Payment method (cash / card / QR / e-money) can be added as a field on PATCH /api/orders/:id; UI would offer a method picker at the "mark paid" step.

c) Menu management

- Table/grid of all items with inline sold-out toggle (PATCH /api/items/:id) — mirrors "sold-out indication" across mobile/table/handy ordering.
- Add/edit item form: name (EN/NE/JA), description, price, emoji/image, category, recommended flag, and options/toppings list — uses POST /api/items and PATCH /api/items/:id.
- Category management: add/reorder categories (POST /api/categories).
- Stretch goal for "Time-based Menu" and "Table-specific Menu": add optional availability windows and table-group tags to items; current schema already has room to extend (a soldOut-style boolean flag pattern).

d) Staff calls panel

- Live list of pending staff calls (GET /api/staff-calls?status=pending), grouped by type (general / water / bill).
- Resolve button per call (PATCH /api/staff-calls/:id) and a Socket.IO listener so new calls pop in without refresh.

e) Sales analytics

- Summary tiles: total sales, total paid orders, average ticket, active orders, pending staff calls — all served by GET /api/analytics/summary.
- Sales-by-day line/bar chart, top products bar chart, and sales-by-category breakdown — rendered with Chart.js (loaded from a CDN in the page).
- This is the "≈30 analysis views" feature area simplified to the handful of views a small restaurant checks daily; documented as an extension point for by-staff and by-hour breakdowns (data already carries timestamps needed for that).

f) Tables overview

- Grid of all tables with status (open / occupied) and a quick link into that table's current bill.

2.3 Shared / cross-cutting polish

- Extend i18n.js with the kitchen and admin strings (currently only customer-facing strings exist).

- Add empty-state and error-state messaging to every list (no orders yet, no staff calls, menu search with no matches).
- Mobile QA pass on the kitchen and admin screens (they will mostly be used on tablets/desktops, but should not break on a phone).
- Basic accessibility pass: focus states (already defined in style.css), colour-contrast check on status pills, aria-labels on icon-only buttons (■, +, −).

2.4 Documentation

- README.md: how to install (npm install), run (npm start), the default port, and the demo admin PIN.
- A short "feature → file" map so a future contributor can find where each USEN-inspired feature lives.
- Notes on what is intentionally simplified for a demo build (JSON file storage instead of a real database, mock payment capture instead of a live payment gateway, no delivery-platform or accounting-software integrations).

2.5 Explicitly out of scope for this build

The following USEN feature groups involve third-party hardware or external platform contracts and are noted here so expectations are clear, rather than being partially faked:

- Real credit-card / QR payment processing (would integrate a payment gateway's SDK).
- Delivery-platform integration (e.g. Uber Eats) — would consume that platform's partner API.
- Physical ticket/receipt printers, kitchen-display hardware, delivery robots, drink-remaining-volume sensors.
- Third-party integrations: attendance systems, security cameras, reservation ledgers, accounting software.

3. Suggested order of work

Step	Task	Depends on
1	Kitchen display page + realtime ticket updates	Existing /api/orders + Socket.IO events (ready)
2	Admin login + orders/POS + tables view	Existing /api/orders, /api/tables (ready)
3	Admin menu management (items, categories, sold-out)	Existing /api/items, /api/categories (ready)
4	Admin staff-calls panel	Existing /api/staff-calls (ready)
5	Admin analytics charts	Existing /api/analytics/summary (ready)
6	i18n additions for kitchen/admin + empty states	Steps 1–5
7	README + feature map + final QA pass	All of the above

Every API endpoint referenced above already exists and is running in server.js — the remaining work is front-end pages and scripts that call them, plus documentation and polish. No backend redesign is required.